

✓ Numpy workbook

Creating a numpy array

✓ Numpy workbook

1. Use `zeros_like`, `ones_like` and `full_like` to create new arrays. Why would you use these?
2. Create a new array by stacking a bunch of arrays. Hint: Use `hstack` and `vstack` for this
3. It is possible to create an array from a list and a tuple (array dimensions are inferred automatically). Can you create an array from a python dict? Why not? Can you modify the python dict in some way OR can you use the keys of the python dict to create the array with feature names? Explore.

```
import numpy as np

# Use zeros_like, ones_like, and full_like to create new arrays.
# These functions are useful for creating new arrays with the same shape and data type
# as an existing array, but with all elements initialized to 0, 1, or a specified fill value,
# respectively. This avoids needing to know the dimensions or dtype of the original array
# beforehand.

original_array = np.array([[1, 2, 3], [4, 5, 6]])

zeros_array = np.zeros_like(original_array)
ones_array = np.ones_like(original_array)
full_array = np.full_like(original_array, 10)

print("Original array:")
print(original_array)
print("\nZeros array:")
print(zeros_array)
print("\nOnes array:")
print(ones_array)
print("\nFull array (filled with 10):")
print(full_array)

# Create a new array by stacking a bunch of arrays using hstack and vstack.
array1 = np.array([1, 2, 3])
array2 = np.array([4, 5, 6])
array3 = np.array([[7], [8], [9]])
array4 = np.array([[10], [11], [12]])

hstack_array = np.hstack((array1, array2))
vstack_array = np.vstack((array1, array2))
hstack_2d_array = np.hstack((array3, array4))

print("\nArray 1:")
print(array1)
print("\nArray 2:")
print(array2)
print("\nHStack of array1 and array2:")
print(hstack_array)
print("\nVStack of array1 and array2:")
print(vstack_array)
print("\nArray 3:")
print(array3)
print("\nArray 4:")
print(array4)
print("\nHStack of array3 and array4:")
print(hstack_2d_array)

# Can you create an array from a python dict? Why not?
# You cannot directly create a NumPy array from a Python dictionary using a single function
# like np.array(). This is because dictionaries do not have a inherent ordered structure
# like lists or tuples, which is necessary for the multidimensional array structure of NumPy.

# Can you modify the python dict in some way OR can you use the keys of the python dict
# to create the array with feature names? Explore.
# While you can't directly convert a dict to a NumPy array in a single step that preserves
# keys as feature names in a standard NumPy array, you can use the dictionary's items
# (keys and values) to construct a structured NumPy array or use libraries like pandas
# which are built on NumPy and can handle labeled data.

# Example of creating a structured array from dictionary items
```

```

data_dict = {'feature1': [10, 20, 30], 'feature2': [1.1, 2.2, 3.3]}
dtype = [('feature1', int), ('feature2', float)]
structured_array = np.array([(data_dict['feature1'][i], data_dict['feature2'][i]) for i in range(len(data_dict['feature1']))], dtype=dtyp

print("\nStructured array from dictionary:")
print(structured_array)
print("\nAccessing data by field name:")
print(structured_array['feature1'])

# Using pandas for labeled data is often more convenient
import pandas as pd
df = pd.DataFrame(data_dict)
print("\nPandas DataFrame from dictionary:")
display(df)

```

Original array:

```
[[1 2 3]
 [4 5 6]]
```

Zeros array:

```
[[0 0 0]
 [0 0 0]]
```

Ones array:

```
[[1 1 1]
 [1 1 1]]
```

Full array (filled with 10):

```
[[10 10 10]
 [10 10 10]]
```

Array 1:

```
[1 2 3]
```

Array 2:

```
[4 5 6]
```

HStack of array1 and array2:

```
[1 2 3 4 5 6]
```

VStack of array1 and array2:

```
[[1 2 3]
 [4 5 6]]
```

Array 3:

```
[[7]
 [8]
 [9]]
```

Array 4:

```
[[10]
 [11]
 [12]]
```

HStack of array3 and array4:

```
[[ 7 10]
 [ 8 11]
 [ 9 12]]
```

Structured array from dictionary:

```
[(10, 1.1) (20, 2.2) (30, 3.3)]
```

Accessing data by field name:

```
[10 20 30]
```

Pandas DataFrame from dictionary:

	feature1	feature2	
0	10	1.1	
1	20	2.2	
2	30	3.3	

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

▼ Array attributes

1. Create an exhaustive list of numpy attributes apart from dim, shape, transpose, ndims and more.
2. What happens when you call `arr.T` in the background? Is a transpose of the array precomputed and stored or does every call to `arr.T` recompute the transpose of the array `arr`.
3. How many ways are there in numpy to change the shape of an array? Are all these methods same or differ in some way?

```
import numpy as np

# Create an exhaustive list of numpy attributes apart from dim, shape, transpose, ndims and more.
# Some common and useful NumPy array attributes include:
# dtype: Data type of the elements in the array.
# size: Total number of elements in the array.
# itemsize: Length of one array element in bytes.
# nbytes: Total bytes consumed by the elements of the array.
# strides: Tuple of bytes to step in each dimension when traversing an array.
# flat: A 1-D iterator over the array.
# real: The real part of the array.
# imag: The imaginary part of the array.
# T: The transposed array. (Mentioned in the prompt, but included for completeness)
# data: Python buffer object pointing to the start of the array's data.
# base: If the array is a view on another array, returns that array, otherwise returns None.

arr = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.int64)

print("Array:")
print(arr)
print("\nAttributes:")
print(f"dtype: {arr.dtype}")
print(f"size: {arr.size}")
print(f"itemsize: {arr.itemsize}")
print(f"nbytes: {arr.nbytes}")
print(f"strides: {arr.strides}")
print(f"flat (first 5 elements): {[next(arr.flat) for _ in range(arr.size)]}")
print(f"real:\n{arr.real}")
print(f"imag:\n{arr.imag}")
print(f"T:\n{arr.T}")
print(f"base: {arr.base}") # Will be None if not a view

# What happens when you call arr.T in the background? Is a transpose of the array precomputed and stored or does every call to arr.T rec
# When you call `arr.T`, a new array is NOT created by default. Instead, `arr.T` returns a VIEW of the original array with the strides a
# This means the data is not copied, and changes to the transposed view will affect the original array (and vice versa) as they share th
# The transpose is essentially "computed" by changing how NumPy interprets the dimensions and strides of the array when accessing element

# Example demonstrating that .T is a view:
arr_original = np.array([[1, 2], [3, 4]])
arr_transpose_view = arr_original.T
print("\nOriginal array before modifying transpose view:")
print(arr_original)
print("\nTranspose view before modification:")
print(arr_transpose_view)

arr_transpose_view[0, 0] = 99 # Modify the view

print("\nOriginal array after modifying transpose view:")
print(arr_original)
print("\nTranspose view after modification:")
print(arr_transpose_view)

# How many ways are there in numpy to change the shape of an array? Are all these methods same or differ in some way?
# There are several ways to change the shape of a NumPy array, and they differ in whether they return a view or a new array (a copy).

# 1. reshape(): This is the most common way. It returns a new array with the desired shape. By default, it tries to return a view if possible.
arr_reshape = arr.reshape(3, 2)
print("\nOriginal array:")
print(arr)
print("\nReshaped array (3x2):")
print(arr_reshape)
print(f"Is reshaped_array a view? {arr_reshape.base is arr}") # Check if it's a view

# 2. resize(): This method changes the shape of the array IN PLACE. It returns None. If the new shape is larger than the original, the array is extended with zeros.
arr_resize = np.array([1, 2, 3, 4, 5, 6])
# print("\nOriginal array before resize:")
# print(arr_resize)
# arr_resize.resize(3, 3) # Reshapes in place
# print("\nArray after resize (3x3):")
# print(arr_resize)

# 3. ravel(): This method returns a 1D flattened view of the array. It will return a view if possible, otherwise a copy.
arr_ravel = arr.ravel()
print("\nOriginal array:")
print(arr)
print("\nRaveled array (flattened view):")
print(arr_ravel)
print(f"Is raveled_array a view? {arr_ravel.base is arr}")

# 4. flatten(): This method always returns a 1D flattened COPY of the array.
```

```

arr_flatten = arr.flatten()
print("\nOriginal array:")
print(arr)
print("\nFlattened array (flattened copy):")
print(arr_flatten)
print(f"Is flattened_array a view? {arr_flatten.base is arr}") # Will be False

# 5. Using the `shape` attribute directly: You can assign a new shape tuple to the `shape` attribute of an array. This is similar to `reshape`.
arr_shape_assign = arr.copy() # Work on a copy to avoid modifying the original 'arr'
arr_shape_assign.shape = (2, 3)
print("\nOriginal array:")
print(arr)
print("\nArray after shape assignment (2x3):")
print(arr_shape_assign)
print(f"Is array after shape assignment a view? {arr_shape_assign.base is arr}")

# Summary of differences:
# - reshape() and assigning to .shape: Return a new array, often a view if possible.
# - resize(): Modifies the array in place.
# - ravel(): Returns a flattened view if possible, otherwise a copy.
# - flatten(): Always returns a flattened copy.

# The choice of method depends on whether you need a new array or want to modify in place,
# and whether you require a view or a copy. Using views is generally more memory-efficient
# when possible.

```



```

Array:
[[1 2 3]
 [4 5 6]]

Attributes:
dtype: int64
size: 6
itemsize: 8
nbytes: 48
strides: (24, 8)
flat (first 5 elements): [np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1)]
real:
[[1 2 3]
 [4 5 6]]
imag:
[[0 0 0]
 [0 0 0]]
T:
[[1 4]
 [2 5]
 [3 6]]
base: None

Original array before modifying transpose view:
[[1 2]
 [3 4]]

Transpose view before modification:
[[1 3]
 [2 4]]

Original array after modifying transpose view:
[[99  2]
 [ 3  4]]

Transpose view after modification:
[[99  3]
 [ 2  4]]

Original array:
[[1 2 3]
 [4 5 6]]

Reshaped array (3x2):
[[1 2]
 [3 4]
 [5 6]]
Is reshaped_array a view? True

Original array:
[[1 2 3]
 [4 5 6]]

Raveled array (flattened view):
[1 2 3 4 5 6]
Is raveled_array a view? True

Original array:

```

▼ Indexing

1. An array has n elements and has shape (p, q) such that $p \times q = n$. Consider row-major storage format. What will be the index of the k th element where $k \leq n$. What will be its index in the column-major storage format?
2. Suppose you have a 4D tensor batch with dimension (b, c, m, n) . The understanding is that it is a collection of b images, each with c channels of size $m \times n$. Suppose you want to access the second channel of the first, fourth and eighth image from batch. Write the indexing for this.

```
import numpy as np

# An array has n elements and has shape (p,q) such that p * q = n.
# Consider row-major storage format. What will be the index of the kth element where k <= n.
# In row-major order, elements of a row are stored contiguously.
# For a 2D array with shape (p, q), the element at row i and column j (0-indexed)
# is located at the index (i * q + j) in a flattened 1D representation.
# If we consider the k-th element (1-indexed, so k starts from 1) in the flattened array,
# its row index (0-indexed) would be (k-1) // q (integer division), and its column index
# (0-indexed) would be (k-1) % q (modulo operation).

# What will be it's index in the column-major storage format?
# In column-major order, elements of a column are stored contiguously.
# For a 2D array with shape (p, q), the element at row i and column j (0-indexed)
# is located at the index (j * p + i) in a flattened 1D representation.
# If we consider the k-th element (1-indexed) in the flattened array,
# its column index (0-indexed) would be (k-1) // p, and its row index
# (0-indexed) would be (k-1) % p.

# Example: Array of shape (2, 3), n=6
# [[1, 2, 3],
#  [4, 5, 6]]
# k=4 (element 4)
# Row-major: (k-1) = 3. Row index: 3 // 3 = 1. Column index: 3 % 3 = 0. Element at (1, 0) is 4.
# Flattened row-major: [1, 2, 3, 4, 5, 6]. Index of 4 is 3 (0-indexed).
# Column-major:
# [[1, 4],
#  [2, 5],
#  [3, 6]] (This is how it would be conceptually stored in column-major for a (2,3) array)
# For the original array with shape (2,3), the element at (1,0) (value 4)
# in column-major flattened representation would be at index (0 * 2 + 1) = 1.
# Let's verify with k=4 (1-indexed) in a flattened column-major view:
# Flattened column-major: [1, 4, 2, 5, 3, 6]. Index of 4 is 1 (0-indexed).
# Column index: (k-1) // p = 3 // 2 = 1. Row index: (k-1) % p = 3 % 2 = 1. This gives (1,1) - doesn't match.
# Let's rethink the k-th element in flattened column-major.
# For k=4, the element is 4. Its original position is (1, 0).
# In column-major, column 0 comes first: [1, 4], then column 1: [2, 5], then column 2: [3, 6].
# Flattened column-major: [1, 4, 2, 5, 3, 6]. The 4th element (k=4) is 5, which is at index 3.
# For the k-th element (1-indexed) in the flattened array, its index in column-major storage corresponds to
# the element at row (k-1) % p and column (k-1) // p in the original array shape (p, q).
# Example: k=4, p=2, q=3. Element is at (4-1)%2 = 1 and (4-1)//2 = 1. Position (1,1) which is 5.
# The index in the flattened column-major array for the element originally at (i, j) (row-major)
# is (j * p + i). For the k-th element in row-major (at row (k-1)//q, col (k-1)%q),
# its index in flattened column-major is (((k-1)%q) * p + ((k-1)//q)).
# Let's test with k=4 (element 4 at row 1, col 0 in row-major):
# Index in flattened column-major: (0 * 2 + 1) = 1. This is the index of element 4 in the column-major flattened array [1, 4, 2, 5, 3, 6].

# Suppose you have a 4D tensor batch with dimension (b, c, m, n).
# The understanding is that it is a collection of b images, each with c channels of size m x n.
# Suppose you want to access the second channel of the first, fourth and eighth image from batch.
# Write the indexing for this.

# Create a dummy 4D tensor for demonstration
b, c, m, n = 10, 3, 32, 32 # 10 images, 3 channels, 32x32 size
batch = np.arange(b * c * m * n).reshape(b, c, m, n)

# Access the second channel (index 1, since 0-indexed) of the first (index 0),
# fourth (index 3), and eighth (index 7) images.
# We can use a list or array of indices for the first dimension (images).
image_indices = [0, 3, 7]
channel_index = 1 # Second channel

# Indexing for the second channel of the first, fourth, and eighth images
# This will return a new array with shape (3, m, n), where 3 is the number of selected images.
selected_channels = batch[image_indices, channel_index, :, :]

print(f"\nShape of the original batch tensor: {batch.shape}")
print(f"Shape of the selected channels tensor: {selected_channels.shape}")

# You can also use boolean indexing if needed, but integer array indexing is suitable here.
# Example of accessing the second channel of the first image:
# print("\nSecond channel of the first image:")
# print(batch[0, 1, :, :])
```

```
# Example of accessing the second channel of the fourth image:
# print("\nSecond channel of the fourth image:")
# print(batch[3, 1, :, :])

# Example of accessing the second channel of the eighth image:
# print("\nSecond channel of the eighth image:")
# print(batch[7, 1, :, :])

# The selected_channels array contains the second channel data for all specified images.
# print("\nSelected channels (second channel of images 0, 3, and 7):")
# print(selected_channels)
```



```
Shape of the original batch tensor: (10, 3, 32, 32)
Shape of the selected channels tensor: (3, 32, 32)
```

✓ Datatypes

1. What happens if you do `np.array(['abc', 'def'])`
2. What happens if you run `arr.astype(int)` when `arr` is a float array? Will the values of the resulting array differ in magnitude beyond machine precision from the original array?
3. What happens if you run `arr.astype(float)` when `arr` is an int array? Will the values of the resulting array differ in magnitude beyond machine precision from the original array?

```
import numpy as np
```

```
# What happens if you do np.array(['abc', 'def'])
# When you create a NumPy array with strings, NumPy will infer the data type
# as a string type (either fixed-length Unicode 'U' or bytes 'S', or object 'O').
# The length of the strings will determine the size of the elements in the array.
# In this case, it will likely create an array of fixed-length Unicode strings.
string_array = np.array(['abc', 'def'])
print("Array created from a list of strings:")
print(string_array)
print(f"Data type: {string_array.dtype}")
print(f"Item size in bytes: {string_array.itemsize}") # Shows the number of bytes for each string element
```

```
# What happens if you run arr.astype(int) when arr is a float array?
# Will the values of the resulting array differ in magnitude beyond machine precision from the original array?
# When you convert a float array to an integer array using astype(int), the fractional part of
# each float number is truncated (地板操作). This means the values in the resulting integer array
# will be the integer part of the original float numbers.
# The values can definitely differ in magnitude significantly from the original float values,
# especially if the original float numbers have large fractional parts. It's not just a matter
# of machine precision; it's a change in the value itself due to the removal of the decimal part.
```

```
float_array = np.array([1.1, 2.9, -3.5, 4.0])
int_array_from_float = float_array.astype(int)

print("\nOriginal float array:")
print(float_array)
print("Integer array after astype(int):")
print(int_array_from_float)
print("Difference (original - converted):")
print(float_array - int_array_from_float) # Shows the difference, highlighting the loss of precision
```

```
# What happens if you run arr.astype(float) when arr is an int array?
# Will the values of the resulting array differ in magnitude beyond machine precision from the original array?
# When you convert an integer array to a float array using astype(float), the integer values
# are converted to their floating-point representation.
# For most common integer types and values that can be exactly represented as floating-point numbers
# (up to a certain magnitude), the values of the resulting array will not differ in magnitude
# beyond machine precision. The conversion is generally exact within the limits of the float type's
# precision. However, for very large integers that exceed the precision limits of the float type,
# some loss of precision might occur, where the exact integer cannot be represented precisely
# as a float.
```

```
int_array = np.array([1, 10, -5, 1000000000000000000]) # Large integer
float_array_from_int = int_array.astype(float)
```

```
print("\nOriginal integer array:")
print(int_array)
print("Float array after astype(float):")
print(float_array_from_int)
# Comparing integer and float representations - for large numbers,
# direct comparison can be tricky due to float representation.
# Let's check if the conversion maintained the value where expected.
```

```
# For smaller numbers, the conversion is exact. For the large number,
# it might lose precision depending on the float type (e.g., float64 vs float32).
# Note: Python's int has arbitrary precision, but NumPy's int types are fixed size.
# Using Python's arbitrary precision int for demonstration of large number behavior.
large_int = 10**20
large_int_np = np.array([large_int], dtype=object) # Use object dtype for arbitrary Python int
large_float_np = large_int_np.astype(float)
print(f"\nLarge Python integer: {large_int}")
print(f"Converted to NumPy float: {large_float_np[0]}")
print(f"Is original large int exactly equal to converted float? {large_int == large_float_np[0]}") # Likely False for very large numbers:
```

↳ Array created from a list of strings:

```
['abc' 'def']
Data type: <U3
Item size in bytes: 12
```

```
Original float array:
[ 1.1  2.9 -3.5  4. ]
Integer array after astype(int):
[ 1  2 -3  4]
Difference (original - converted):
[ 0.1  0.9 -0.5  0. ]
```

```
Original integer array:
[          1          10          -5
 1000000000000000000]
Float array after astype(float):
[ 1.e+00  1.e+01 -5.e+00  1.e+18]
```

```
Large Python integer: 1000000000000000000
Converted to NumPy float: 1e+20
Is original large int exactly equal to converted float? True
```

✓ Miscellaneous Topics

1. Why would one ever use `new_arr = arr.copy()` when they can simply do `new_arr = arr`.
2. How is view different from reshape?
3. Will view update `arr.shape`. Will reshape update it?
4. Which sorting algorithm runs in the backend when you do `arr.sort(ascending=True)`.

```
import numpy as np
```

```
# Why would one ever use new_arr = arr.copy() when they can simply do new_arr = arr.
# When you do `new_arr = arr`, you are creating a reference to the original array `arr`.
# This means both `new_arr` and `arr` point to the same underlying data in memory.
# If you modify `new_arr`, the changes will also be reflected in `arr`, and vice versa.
#
# When you use `new_arr = arr.copy()`, you are creating a completely new array with its
# own copy of the data from `arr`. Changes made to `new_arr` will NOT affect `arr`,
# and changes made to `arr` will NOT affect `new_arr`.
#
# You would use `copy()` when you need to modify an array without altering the original array.
```

```
arr = np.array([1, 2, 3])
arr_ref = arr
arr_copy = arr.copy()
```

```
print(f"Original array: {arr}")
print(f"Reference array: {arr_ref}")
print(f"Copy array: {arr_copy}")
```

```
arr_ref[0] = 99 # Modify the reference
print(f"\nOriginal array after modifying reference: {arr}")
print(f"Reference array after modifying reference: {arr_ref}")
print(f"Copy array after modifying reference: {arr_copy}") # Copy remains unchanged
```

```
arr_copy[0] = 100 # Modify the copy
print(f"\nOriginal array after modifying copy: {arr}") # Original remains unchanged
print(f"Reference array after modifying copy: {arr_ref}") # Reference also remains unchanged
print(f"Copy array after modifying copy: {arr_copy}")
```

```
# How is view different from reshape?
# A **view** is a new array object that looks at the SAME data as the original array.
# It does not own the data itself. Changes to the view affect the original array,
# and changes to the original array affect the view. Views are created by operations
# like slicing (arr[start:end]), transposing (arr.T), and sometimes reshaping
# if the new shape is compatible with the original memory layout without requiring a data copy.
#
# **Reshape** is a method that returns a new array with a different shape.
```

```

# While `reshape` *can* return a view if the new shape is compatible with the original
# memory layout (e.g., changing strides), it will return a COPY of the data if the
# new shape requires a different memory layout (e.g., changing the order of elements
# in memory, like from C-order to Fortran-order). You can control this behavior
# with the `order` parameter in `reshape`.

arr_original = np.array([[1, 2], [3, 4]])
arr_view = arr_original[:] # Create a view using slicing
arr_resized = arr_original.reshape(4, 1) # Reshape - might be a view or copy

print("\nOriginal array:")
print(arr_original)
print("\nView of original array:")
print(arr_view)
print("\nResized array:")
print(arr_resized)

print(f"\nIs arr_view a view? {arr_view.base is arr_original}")
print(f"Is arr_resized a view? {arr_resized.base is arr_original}") # Check if reshape returned a view

arr_view[0, 0] = 99 # Modify the view
print("\nOriginal array after modifying view:")
print(arr_original)

# If arr_resized was a view, modifying it would affect arr_original.
# If arr_resized was a copy, modifying it would not affect arr_original.
# Let's check by modifying arr_resized:
# arr_resized[0, 0] = 100
# print("\nOriginal array after modifying resized array:")
# print(arr_original) # Will be affected if arr_resized was a view

# Will view update arr.shape. Will reshape update it?
# Neither a view nor `reshape` will update the `shape` attribute of the *original* array (`arr`).
#
# A **view** is a new array object with its own attributes, including `shape`.
# The `shape` of the view will reflect the dimensions of the view itself, not the original array.
# The original array's `shape` remains unchanged.
#
# `**reshape**` returns a *new* array object with the specified shape. The `shape` attribute
# of this *new* array will be the shape you provided. The `shape` attribute of the *original*
# array remains unchanged.
#
# The `resize()` method, however, *does* change the shape of the array IN PLACE,
# thus updating the `shape` attribute of the original array.

arr = np.array([[1, 2], [3, 4]])
print(f"\nOriginal array shape before view/reshape: {arr.shape}")

arr_view = arr[:]
print(f"Shape of the view: {arr_view.shape}")
print(f"Original array shape after creating view: {arr.shape}") # Original shape is unchanged

arr_resized = arr.reshape(4, 1)
print(f"Shape of the resized array: {arr_resized.shape}")
print(f"Original array shape after resizing: {arr.shape}") # Original shape is unchanged

# Using resize (demonstrates in-place shape change)
# arr_to_resize = np.array([1, 2, 3, 4])
# print(f"\nArray before resize shape: {arr_to_resize.shape}")
# arr_to_resize.resize(2, 2)
# print(f"Array after resize shape: {arr_to_resize.shape}") # Original array's shape is updated

# Which sorting algorithm runs in the backend when you do arr.sort(ascending=True).
# NumPy's `arr.sort()` method (and `np.sort()`) uses different sorting algorithms
# depending on the data type and potentially the version of NumPy.
#
# By default, for numerical types, NumPy often uses **quicksort**. However, it can
# fall back to other algorithms like **mergesort** or **heapsort** in certain situations
# (e.g., for stability or specific data characteristics).
#
# The `kind` parameter in `arr.sort()` allows you to explicitly specify the sorting algorithm:
# - 'quicksort'
# - 'mergesort'
# - 'heapsort'
# - 'stable' (uses a stable sort, typically mergesort or Timsort depending on version)
#
# The `ascending=True` part is not a standard parameter for `arr.sort()`.
# `arr.sort()` sorts in ascending order by default. To sort in descending order,
# you would typically sort in ascending order and then reverse the array,
# or use slicing `[::-1]` on the sorted array.

```



```

arr_sort_example = np.array([3, 1, 4, 1, 5, 9, 2, 6])
arr_sort_example.sort() # Sorts in place (ascending by default)
print(f"\nArray after arr.sort(): {arr_sort_example}")

# To get descending order:
arr_descending = np.array([3, 1, 4, 1, 5, 9, 2, 6])
arr_descending.sort()
arr_descending = arr_descending[::-1]
print(f"Array sorted in descending order: {arr_descending}")

# Using np.sort() which returns a new sorted array:
arr_original_for_np_sort = np.array([3, 1, 4, 1, 5, 9, 2, 6])
arr_np_sorted = np.sort(arr_original_for_np_sort) # Returns a new sorted array
print(f"New array from np.sort(): {arr_np_sorted}")

# Explicitly specifying sort kind:
# arr_quicksort = np.array([3, 1, 4, 1, 5, 9, 2, 6])
# arr_quicksort.sort(kind='quicksort')
# print(f"Array sorted with quicksort: {arr_quicksort}")

```

```

↔ Original array: [1 2 3]
Reference array: [1 2 3]
Copy array: [1 2 3]

Original array after modifying reference: [99 2 3]
Reference array after modifying reference: [99 2 3]
Copy array after modifying reference: [1 2 3]

```

```

Original array after modifying copy: [99 2 3]
Reference array after modifying copy: [99 2 3]
Copy array after modifying copy: [100 2 3]

```

```

Original array:
[[1 2]
 [3 4]]

```

```

View of original array:
[[1 2]
 [3 4]]

```

```

Reshaped array:
[[1]
 [2]
 [3]
 [4]]

```

```

Is arr_view a view? True
Is arr_resaped a view? True

```

```

Original array after modifying view:
[[99 2]
 [ 3 4]]

```

```

Original array shape before view/reshape: (2, 2)
Shape of the view: (2, 2)
Original array shape after creating view: (2, 2)
Shape of the reshaped array: (4, 1)
Original array shape after reshaping: (2, 2)

```

```

Array after arr.sort(): [1 1 2 3 4 5 6 9]
Array sorted in descending order: [9 6 5 4 3 2 1 1]
New array from np.sort(): [1 1 2 3 4 5 6 9]

```

✓ Math

1. When you do `np.exp(arr)` when `arr` is a numpy tensor, then is it doing the matrix exponential e^{arr} or elementwise exponential? A similar question can be asked for `np.log(arr)`.
2. If you know matrix exponentials from your linear algebra, can you even do it in numpy directly?

```

import numpy as np
import numpy.linalg as la # Import linear algebra module for matrix exponential

# When you do np.exp(arr) when arr is a numpy tensor, then is it doing the matrix exponential e^arr or elementwise exponential?
# A similar question can be asked for np.log(arr).
#
# NumPy's `np.exp()` and `np.log()` functions perform ELEMENT-WISE operations.
# `np.exp(arr)` calculates the exponential of each element in the array `arr`.
# `np.log(arr)` calculates the natural logarithm of each element in the array `arr`.
# They do NOT perform matrix exponential or matrix logarithm operations.

elementwise_array = np.array([[1, 2], [3, 4]])

```

```
exp_elementwise = np.exp(elementwise_array)
log_elementwise = np.log(elementwise_array)

print("Original array:")
print(elementwise_array)
print("\nElement-wise exponential (np.exp):")
print(exp_elementwise)
print("\nElement-wise natural logarithm (np.log):")
print(log_elementwise)

# If you know matrix exponentials from your linear algebra, can you even do it in numpy directly?
# Yes, you can perform matrix exponential in NumPy, but you need to use functions from the
# `numpy.linalg` module. The function for calculating the matrix exponential is `scipy.linalg.expm`.
# Note: While NumPy has a `linalg` module, the matrix exponential function is often found
# in `scipy.linalg`. If you don't have SciPy installed, you might need to install it (`pip install scipy`).

# For demonstrating, let's assume scipy is available or use a simpler example if not.
# If scipy is available:
try:
    from scipy.linalg import expm
    matrix = np.array([[0, 1], [0, 0]])
    matrix_exponential = expm(matrix)
    print("\nMatrix for exponential:")
    print(matrix)
    print("\nMatrix exponential (using scipy.linalg.expm):")
    print(matrix_exponential)

except ImportError:
    print("\nSciPy not installed. Cannot demonstrate matrix exponential directly.")
    print("To perform matrix exponential, you typically need scipy.linalg.expm.")

# NumPy's own linalg.matrix_power can be used for integer matrix powers, but not the exponential.
# For matrix logarithm, you would typically use `scipy.linalg.logm`.
```

```
↺ Original array:
[[1 2]
 [3 4]]

Element-wise exponential (np.exp):
[[ 2.71828183  7.3890561 ]
 [20.08553692 54.59815003]]

Element-wise natural logarithm (np.log):
[[0.         0.69314718]
 [1.09861229 1.38629436]]

Matrix for exponential:
[[0 1]
 [0 0]]

Matrix exponential (using scipy.linalg.expm):
[[1. 1.]
 [0. 1.]]
```

Implement the following functions using numpy. $\mathbf{x} \in \mathbb{R}^n$.

1. $f(\mathbf{x}) = \mathbf{x}^2$
2. $f(\mathbf{x}) = \frac{1}{1+e^{-x}}$
3. $f(\mathbf{x}) = \max(\mathbf{0}, \mathbf{x})$
4. $f(\mathbf{x}) = \log(\sum_{x_i \in \mathbf{x}} e^{x_i})$.

```
import numpy as np
```

```
# Implement the following functions using numpy. x is a numpy array.
```

```
# 1. f(x) = x^2 (element-wise squaring)
def f1(x):
    return x**2
```

```
# 2. f(x) = 1 / (1 + e^(-x)) (sigmoid function)
def f2(x):
    return 1 / (1 + np.exp(-x))
```

```
# 3. f(x) = max(0, x) (ReLU function)
def f3(x):
    return np.maximum(0, x)
```

```
# 4. f(x) = log(sum(e^xi)) (Log-Sum-Exp function)
# This function is often used to compute the logarithm of the sum of exponentials
# of the elements of a vector or matrix in a numerically stable way.
```

```

def f4(x):
    # For numerical stability, it's better to use scipy.special.logsumexp
    # but implementing directly as requested:
    return np.log(np.sum(np.exp(x)))

# Example usage:
x_vector = np.array([1.0, -2.0, 0.5])
x_matrix = np.array([[1.0, -2.0], [0.5, 3.0]])

print("Input vector x:", x_vector)
print("f1(x) (element-wise squaring):", f1(x_vector))
print("f2(x) (sigmoid):", f2(x_vector))
print("f3(x) (ReLU):", f3(x_vector))
print("f4(x) (Log-Sum-Exp):", f4(x_vector))

print("\nInput matrix x:\n", x_matrix)
print("f1(x) (element-wise squaring):\n", f1(x_matrix))
print("f2(x) (sigmoid):\n", f2(x_matrix))
print("f3(x) (ReLU):\n", f3(x_matrix))
print("f4(x) (Log-Sum-Exp for matrix - sums all elements):", f4(x_matrix))

```

```

→ Input vector x: [ 1. -2.  0.5]
f1(x) (element-wise squaring): [1.  4.  0.25]
f2(x) (sigmoid): [0.73105858 0.11920292 0.62245933]
f3(x) (ReLU): [1.  0.  0.5]
f4(x) (Log-Sum-Exp): 1.5045969023422836

Input matrix x:
[[ 1. -2. ]
 [ 0.5  3. ]]
f1(x) (element-wise squaring):
[[1.  4. ]
 [0.25 9. ]]
f2(x) (sigmoid):
[[0.73105858 0.11920292]
 [0.62245933 0.95257413]]
f3(x) (ReLU):
[[1.  0. ]
 [0.5  3. ]]
f4(x) (Log-Sum-Exp for matrix - sums all elements): 3.2022534476793516

```